

Saturday

Date: 07-03-2026

Mock Test

#

Sunday

Date: 08-03-2026

Holiday

#

Scope

Date: 09-03-2026

-- Local scope --

--- A variable in the local scope when it is defined inside a function.--
'''

```
def func():  
a=10      # -- Local scope variable  
print(a)  
func()  
'''
```

#print(a) -- NameError: name 'a' is not defined

-- Global Scope --

A variable is a global scope when it is defined at the top level of the module.

Accessible throughout the program unless
#--shared by a local variable with the same name.

if you want to modify a global variable inside function
#--you must use the global keyword
'''

```
a=7        # --- global scope variable  
def glob():  
a=10  
print(a)  
print(a)   # -- global will call  
glob()  
print(a)   # -- same  
'''
```

'''

a=7

```
def gl():
    global a
    print(a)    ## -- 7
    a=10
    print(a)    ## -- 10
    gl()
```

```
print(a)    ## -- 10 ?
```

```
'''
```

```
### -- Enclosed Scope --- :
```

```
# Enclosing scope refers to the variable in the outer function.
```

```
## The inner function has the accessible variable of the enclosing scope.
```

```
### If you want to modify the enclosing variable inside the
#--- inner function you must use the nonlocal keyword.
```

```
'''
```

```
def outer():
    a=10
    def inner():
        a=5
        print(a)
        print(a)
        inner()
        print(a)
    outer()
```

```
print("-----")
```

```
'''
```

```
'''
```

```
def outer():
    a=10
    def inner():
        nonlocal a
        a=5
        print(a)
        print(a)
        inner()
        print(a)
    outer()
```

```
'''
```

```
#-----
```

```
# Decorator --- @ funcName
```

```
# Need to take a function as a parameter
```

```
## Add functionality to the function
```

Function need to return another function

```
"""
The decorator is a function that adds functionality to the another function
"without modifying the code"
"""
```

```
# Normal
```

```
"""
def outer(a):
def inner():
return a.upper()
return inner

def func():
return "bankai"
print(outer(func())())
"""
```

```
# using @outer keyword
```

```
"""
def outer(a):
def inner():
return a().upper()
return inner
@outer
def func():
return "hado 99"
print(func())
"""
```

```
# 0 division error
```

```
"""
def outer(a):
def inner(m,n):
if n == 0:
return "Zero error"
return a(m,n)
return inner
@outer
def func(x,y):
return x/y
print(func(10,2))
print(func(10,0))
"""
```

```
# here no changes occurs only check so this can works:
```

```
"""
def outer(a):
def inner(x,y):
if y == 0:
return "Zero error"
return a(x,y)
return inner
@outer
```

```
def func(x,y):
    return x/y
print(func(10,2))
print(func(10,0))
"""
```

```
# 2'times decorater:
"""
```

```
def outer(a):
    def inner():
        return a().upper()
    return inner
```

```
def x(a):
    def y():
        return a().split()
    return y
```

```
@x          # 2nd
@outer      # 1st # calling dow - upwards
```

```
def func():
    return "Yokozo"
print(func())
```

```
"""
```

```
# A_P_Arg
"""
```

```
def outer(x):
    def inner(*args):
        s=args[1:]
        for i in s:
            if i==0:
                return "zeroDivisionError"
        return x(*args)
    return inner
@outer
def func(a,b):
    return a/b
print(func(10,2))
print(func(10,0))
```

```
@outer
def four(a,b,c,d):
    return a/b/c/d
print(four(10,5,2,1))
print(four(10,5,2,0))
"""
```

```
#
```

An iterator is a object that allows you to loopover the element
#- "One at a time using next()".

It is used to traverse the through a sequence.

```
"""
a=[1,2,3,4,5]

b=iter(a)
print(next(b))
print(next(b))
print(next(b))
print(next(b))
print(next(b))
# print(next(b))      # -- StopIteration
"""
```

Dunder or Magic methods --- "__"

```
"""
a=[1,2,3,4,5]
b=a.__iter__()
print(b.__next__())
print(b.__next__())
print(b.__next__())
print(b.__next__())
print(b.__next__())
# print(b.__next__())  # -- StopIteration
"""
```

for loop Iterator

```
"""
a="python"
b=iter(a)
for i in b:
    print(i)
"""
```

#set Iterator

```
"""
a={10,20,30,40}
b=iter(a)
```

```
print(next(b))
print(next(b))
print(next(b))
print(next(b))
"""
```

Dictionary Iterator

Keys

```
"""
a={1:"Aizen",2:"Ichigo",3:"urahara"}
b=iter(a)
```

```

print(next(b))
print(next(b))
print(next(b))
"""

# keys()
"""

a={1:"Aizen",2:"Ichigo",3:"urahara"}
b=iter(a.keys())

print(next(b))
print(next(b))
print(next(b))

"""

#Values()
"""

a={1:"Aizen",2:"Ichigo",3:"urahara"}
b=iter(a.values())

print(next(b))
print(next(b))
print(next(b))
"""

#Items()
"""

a={1:"Aizen",2:"Ichigo",3:"urahara"}
b=iter(a.items())
print(next(b))
print(next(b))
print(next(b))
"""

```

Generator

```

# It is a special type of iterator
## used to yield keyword
### Its a lazy evaluation
#### Memory efficient

```

```

# return
"""

```

```

def func():
return 'Bankai'
return 'Shikkai'
print(func())
"""

```

yield

```

# Here Bankai yield only
"""

```

```

def func():
yield 'Bankai'
yield 'Shikkai'

```

```
print(next(func()))
print(next(func()))
"""
# here both will yield
"""
```

```
def func():
yield 'Bankai'
yield 'Shikkai'
b=func()
print(next(b))
print(next(b))
"""
```

```
# yield and ieteration
"""
```

```
def func():
print("Start")
yield 'Bankai'
print("Middle")
yield 'Shikkai'
print("End")
b=func()
print(next(b))    #--op    # Start
# Bankai
```

```
print(next(b))    #--op    # Start
# Bankai
# Middle
# Shikkai
```

```
##print(next(b))    #--op    # End    But also -- error : Stoplteration
"""
```

```
#
"""
```

```
def func(a):
for i in range(1,a):
if i%2==0:
yield i
```

```
for a in func(10):
print(a)
"""
```

```
#
"""
```

```
def fib(n):
aa,b=0,1
for i in range(n):
yield a
a,b = b,a+b
for i in fib(10):
print(i)
"""
```

```
#
"""
```

```
def func():
    for i in [1,2,3]:
        yield i
    b=func()
    for i in b:
        print(i)
    """
```

yield from

```
"""
def func():
    yield from [1,2,3]
    b=func()
    for i in b:
        print(i)
    """
```

#-----

Call by value -- (immutable)

A copy of the reference is passed

The function cannot modify the original object

It can only create new object and doesn't affect original

```
#
"""
```

```
def value(v):
    print("n",id(v))
    n+=5
    print("n modify",id(v))
    a=10
    print("a",id(a))
    value(a)
    print(" a",id(a))
    """
```

Call by reference -- (Mutable)

The function receives the reference of the same object

Changes made to the object inside the function are reflected

outside of the function -- affecting original

```
"""
```

```
a=[1,2,3,4,5]
print(id(a))
a.append(6)
print(id(a))
"""
```

using function

```
"""
```

```
def reference(r):
    print(id(r))
    print(r)
```



```

a.append(6)
print(r)
print(id(r))
a=[1,2,3,4,5]
print(id(a))
reference(a)
print(id(a))
"""

```

```
#
```

```
#      Module                                Date: 11-03-2026
```

```
# Module is a file that contain python code.
#-such s function,variable and class
```

```
## Module helps to organize code and make it easier to value code.
"""

```

```

from . import module_name: Imports from the current package/directory.
from .. import module_name: Imports from the parent directory/package.
from ... import module_name: Imports from the grandparent directory/package.
"""

```

```
# same folder:
```

```

# from file_name import func_name
# -- To call the the function from another file
# * means calling all function
"""

```

```

from Mod import func
print(func())
"""

```

```
## Different Folder:
```

```
# from file_name import func_name
```

```

"""
from ..Leetcode.Leetcode import Solution
print(Solution().myPow(2.000,10))
"""

```

```

import math
#math.ceil() --- op will rounding number to the next number
"""

```

```

a=7.1
print(math.ceil(a))
"""

```

```
#math.floor() op will rounding number to the same value
```

```
"""
```

```

a=7.1
print(math.floor(a))

```

```

"""

#math.pow()
"""
a=7
b=2
print(math.pow(7,2))
"""

#math.sqrt()
"""
a=25
print(math.sqrt(a))
"""

#math.pi()
"""
print(math.pi)
"""

#math.gcd() -- Gretest Common Divisor
"""
a=16
b=8
print(math.gcd(a,b))
"""

import random
# random() between 0 - 1 float values
"""
a=random.random()
print(a)
"""

#random.randint()
"""
b=random.randint(0,2)
print(b)
"""

#random.uniform()
"""
a=random.uniform(1,10)
print(a)
"""

#OTP Generator
"""
a=""
for i in range(6):
a=a+str(random.randint(0,9))
print(a)
"""

# Choice
"""
a=['Ace','Sabo','Luffy']
print(random.choice(a))
"""

# Shuffle -- affecting original
"""

```

```
a=[1,2,3,4,5]
random.shuffle(a)
print(a)
"""
```

```
# sample
"""
```

```
a=['Ace','Sabo','Luffy']
print(random.sample(a,2))
"""
```

```
# Rock papper and Scissors
"""
```

```
humanChoice = int(input("Player Choice"))
computerChoice = random.randint(1,3)
print(computerChoice)
rock = 1
paper = 2
scissor =3
```

```
def sps():
if humanChoice == 1 and computerChoice == 3:
print("Player Win")
elif humanChoice == 2 and computerChoice == 1:
print("Player Win")
elif humanChoice == 3 and computerChoice == 2:
print("Player Win")
else:
print("computer win")
if humanChoice == computerChoice:
print("draw")
else:
sps()
"""
```

```
# Alternate Method
"""
```

```
player=input("rock, paper, scissor : ").lower()
a=["rock","paper","scissor"]
computer=random.choice(a)
```

```
if player in a:
print("player choice",player)
print("computer choice",computer)
if player == computer:
print("draw")
elif (player=="rock" and computer=="scissor") or (player=="paper" and computer=="rock") or
(player=="scissor" and computer=="paper"):
print("Player win")
else:
print("Computer win")
else:
print("Invalid input")
"""
```

```

### game

##player = 0
##computer = 0

'''
score = 0
while True:
    player=input("rock, paper, scissor : ").lower()
    a=["rock","paper","scissor"]
    computer=random.choice(a)

    if player in a:
        print("player choice",player)
        print("computer choice",computer)
        if player == computer:
            print("draw")
            score = score
        elif (player=="rock" and computer=="scissor") or (player=="paper" and computer=="rock") or
            (player=="scissor" and computer=="paper"):
            print("Player win")
            score = score + 1
        else:
            print("Computer win")
            score = score-1
        print("Player Score : ",score)

    else:
        print("Invalid input")
        n=input("paly : 1 , play : 0 :")
        if n != "1":
            break
'''

## date and time :

## Current date and time -- datetime.datetime.now()
'''
import datetime
x = datetime.datetime.now()
print(x)
'''

## strftime
'''
print(x.strftime("%h"))    # -- month name only three letters
print(x.strftime("%H"))    # -- hour
print(x.strftime("%m"))    # -- month in int
print(x.strftime("%M"))    # -- minutes
print(x.strftime("%S"))    # -- Seconds
print(x.strftime("%d"))    # -- date only
print(x.strftime("%D"))    # -- date/month/year(last 2)
print(x.strftime("%a"))    # -- weekday first 3 letters
print(x.strftime("%A"))    # -- weekday
print(x.strftime("%W"))
'''

```

```
## year
"""
print(x.year)
## month
print(x.month)
## day
print(x.year)
## hour
print(x.hour)
## minute
print(x.minute)
## second
print(x.second)
## today date
print(x.day)
"""
#-----
```

```
## regular Expression :
import re
```

```
#findall
"""
a="bankai katenkyokotsu karaamatsu shinju"
b=re.findall("bankai",a)
print(b)
"""
```

```
## search
"""
a="bankai katenkyokotsu karaamatsu shinju"
b=re.search("katen",a)
print(b)
print(b.start())
print(b.end())
print(b.string)
print(b.span())
"""
```

```
#
```

```
# Date: 12-03-2026
```

```
"""
a="aizen@gmail.com"
b=re.findall("@",a)
print(b)
"""
```

```
# password check:
```

```
"""
a=input("Enter your Email id : ")
```

```
if re.findall("@",a) and re.findall(".com",a):
    print("Valid")
else:
    print("Invalid")
'''
```

```
# re.split()
'''
```

```
a="hado no kuju kyu"
b=re.split("o",a)
print(b)
```

```
a="hado no kuju kyu"
b=re.split("k",a)
print(b)
```

```
a="hado no kuju kyu"
b=re.split("u",a)
print(b)
```

```
a="hado no kuju kyu"
b=re.split(" ",a)
print(b)
'''
```

```
#sub
'''
```

```
a="Bankai senbon zakura kageyoshi"
b=re.sub("Bankai","Shikkai",a)
print(b)
```

```
a="Bankai senbon zakura kageyoshi"
b=re.sub(" ","-",a)
print(b)
'''
```

```
# ^ start
```

```
'''
```

```
a="Bankai senbon zakura kageyoshi"
b=re.findall("^Ban",a)
print(b)
'''
```

```
'''
```

```
a="91-1234567890"
b=re.findall("^91-",a)
print(b)
'''
```

```
#
```

```
'''
```

```
a=input()
if re.findall("^91-",a):
    print("Indaian Number")
```

```

else:
print("Other country")
'''

# $ end
'''

a="Bankai senbon zakura kageyoshi"
b=re.findall("kageyoshi$",a)
print(b)
'''

```

```

'''

a=input("enter your E-mail : ")
if re.findall(".com$",a):
print("valid E-mail")
else:
print("Invalid E-mail")
'''

```

```

#

```

```

# Date: 13-03-2026

```

```

'''

name1 = list(input("Enter Your name: ").lower().replace(" ", ""))
name2 = list(input("Enter Your partner name: ").lower().replace(" ", ""))

```

```

for i in name1:
if i in name2:
name1.remove(i)
name2.remove(i)
count = len(name1)+len(name2)

```

```

flames=['f','l','a','m','e','s']
while len(flames)>1:
index = count%len(flames)-1
if index >0:
right =flames[index+1:]
left = flames[:index]
flames = right+left
else:
flames.pop()
result = flames[0]
print(result)
'''

```

```

#_____
import re

```

```

# To print Only num

```

```
"""
a="Phone num : 12345 67890"
b=re.findall("\\d",a)
c=re.findall("\\d+",a)
print(b,c)
"""
```

differentiate the alphabets

```
"""
a="aizen@gmail.com"
b=re.findall("[a-zA-Z]+",a)
print(b)
"""
```

```
# x.
"""
```

```
a="bleach"
b=re.findall("b.",a)
print(b)
"""
```

```
# .x
"""
```

```
a="bleach"
b=re.findall(".h",a)
print(b)
"""
```

```
# x.x
#empty
"""
```

```
a="bleach"
b=re.findall("b.l",a)
print(b)
"""
```

```
# bl
"""
```

```
a="bleach"
b=re.findall("b.e",a)
print(b)
"""
```

```
#x.xx -- blea
"""
```

```
a="bleach"
b=re.findall("b.ea",a)
print(b)
"""
```

```
#_____
```

```
# + required 1 or more char if not list will be empty
"""
```

```
a="bleach"
b=re.findall("b.+e",a)
print(b)
"""
```

```
# empty
```



```

"""
a="bleach"
b=re.findall("b.+l",a)
print(b)
"""

# double str words
"""
a="bleach bankai"
b=re.findall("b.+",a)
print(b)
"""

# _____ -
# * required 0 or more char here not
"""
a="bleach"
b=re.findall("b.*l",a)
print(b)
"""

# bleach
"""
a="bleach"
b=re.findall("b.*h",a)
print(b)
"""

# double str words
"""
a="bleach bankai"
b=re.findall("b.*",a)
print(b)
"""

#
"""
a="bleach bankai"
b=re.findall("b.*a",a)
print(b)
"""

# _____

# ? 0 or 1 char required -- if it is in or not to check
"""
a= "bleach bankai"
b= re.findall("[b.?]",a)
print(b)
"""

#
"""

"""

# {} -- speecified char
"""
a= "bleach bankai"
b= re.findall("[.{a}]",a)
print(b)

```

```

"""
# - check the chars are in or not
"""

a= "bleach bankai"
b= re.findall(".{3}a",a)
print(b)
"""

# | -- like or gate
"""

a= "bleach bankai"
b= re.findall("[bleach|naruto]+",a)
print(b)

a= "bleach bankai"
b= re.findall("[bleach|bankai]+",a)
print(b)
"""

# [a-z]
"""

a= "bleach bankai"
b= re.findall("[a-z]+",a)
print(b)
"""

# [A-Z]

# E-mail
"""

a= "bleach@gmail.com"
b= re.findall("[a-z]+@[a-z].+",a)
print(b)
"""

# r"d" -- raw string no error or \\ will work no error
"""

a= "abc12334445"
b= re.findall(r"/d+",a)
print(b)
"""

#password checking
"""

a= input("Enter your password :")

if len(a)>7:
if re.findall("[!@#$%^&*,./?]",a) == []:
print("Give atleast a one special char")
elif re.findall("[A-Z]",a) == []:
print("Give atleast a one capital letter ")
elif re.findall("[0-9]",a) == []:
print("Give atleast a one integer")

```

```

else:
print("valid password")
else:
print("Invalid")
"""

# using search
"""

a= input("Enter your password :")

if len(a)>7:
if re.search("[!@#$%^&*,./?]",a) is None:
print("Give atleast a one special char")
elif re.search("[A-Z]",a) is None:
print("Give atleast a one capital letter ")
elif re.search("[0-9]",a) is None:
print("Give atleast a one integer")
else:
print("valid password")
else:
print("Invalid")
"""

```

```

# password checking untill correct

```

```

"""

while True:
a= input("Enter your password :")

if len(a)>7:
if re.search("[!@#$%^&*,./?]",a) is None:
print("Give atleast a one special char")
elif re.search("[A-Z]",a) is None:
print("Give atleast a one capital letter ")
elif re.search("[0-9]",a) is None:
print("Give atleast a one integer")
else:
print("valid password")
break
else:
print("Invalid")
"""

```

```

#

```

```

#
Date: 14-03-2026

```

```

# Saturday -- Mock Test
#

```

#

Date: 15-03-2026

sunday -- Holiday